TATA TECHNOLOGIES LTD.

TECH PULSE FY-26 | AI/ML Capstone Project

TECHNICAL REPORT

Field	Details
Institute / University	Pimpri Chinchwad College of Engineering (PCCOE)
Programme / Batch	CS / AIML
Student Name	Ayush Tatya Ahire
PRN	122B1C001
Case Study ID (CS1-CS5)	CS3
Project Title	Automotive Cybersecurity TARA Assistant

1. Problem Statement and Intended Users

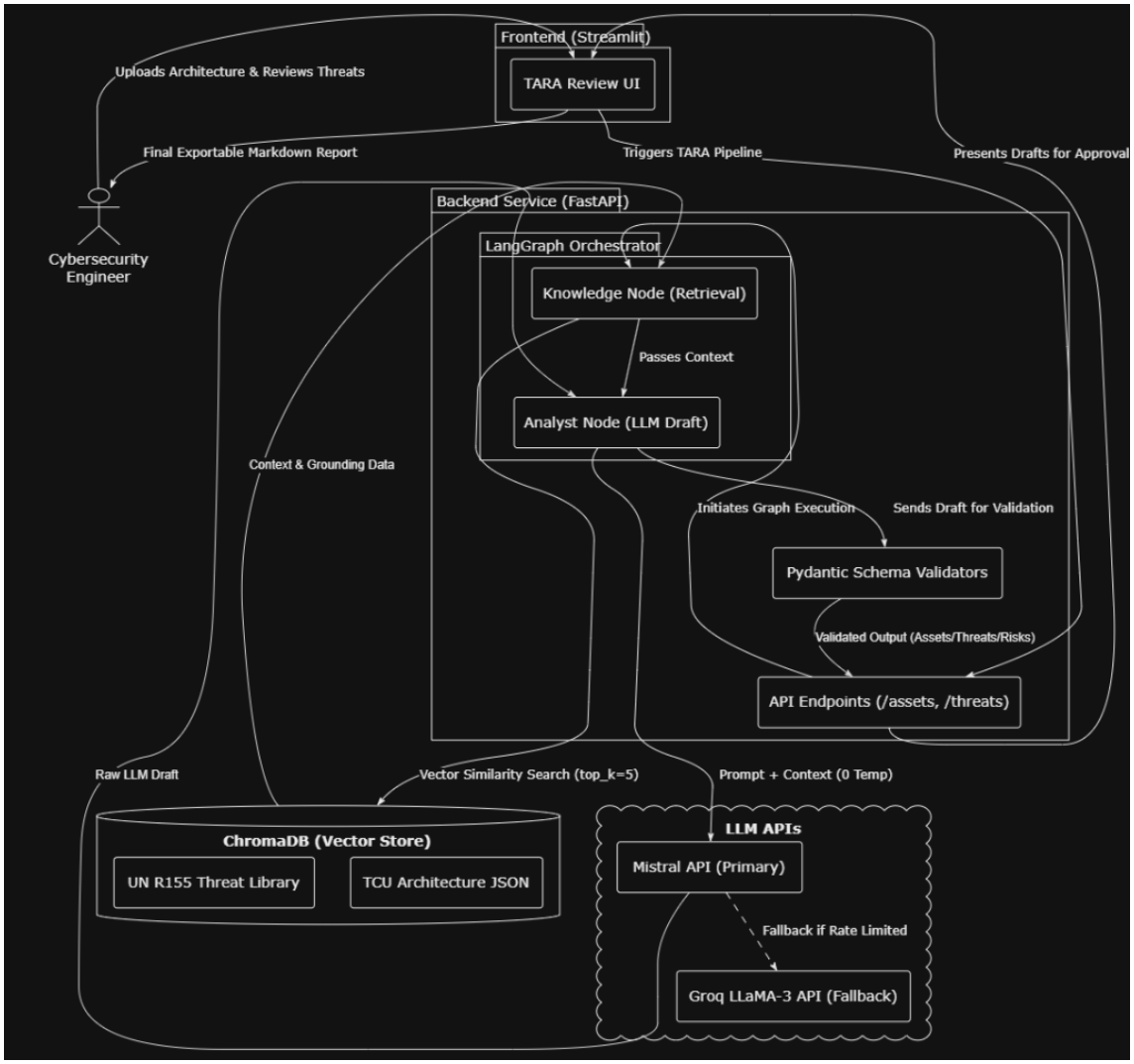
Threat Analysis and Risk Assessment (TARA) is a highly manual, time-consuming process. The intended users are automotive cybersecurity engineers who require an AI assistant to accelerate ISO/SAE 21434 compliance without compromising on deterministic regulatory grounding.

2. Input Data / Knowledge Base

Item	Description
Sources	UN R155 Threat library JSON, TCU Architecture JSON.
Size and format	Structured JSON, small dataset (<1MB).
Preprocessing / EDA done	Loaded directly into RAG Store (ChromaDB) on application startup.
Chunking strategy	Whole record mapping (each threat pattern is an independent document).

3. Solution Architecture

Component	Choice (name, version, hosting)
Pre-trained model	mistral-small-latest (Primary) / Groq LLaMA-3 (Fallback)
Embedding model	BAAI/bge-small-en-v1.5
Vector store (FAISS / ChromaDB)	ChromaDB
RAG pipeline / AI-agent design	LangGraph nodes separating retrieval (Knowledge) and generation (Analyst).
Service layer and UI	FastAPI, Streamlit.
Tools / deterministic rules	Python, Pydantic, HTTPX.

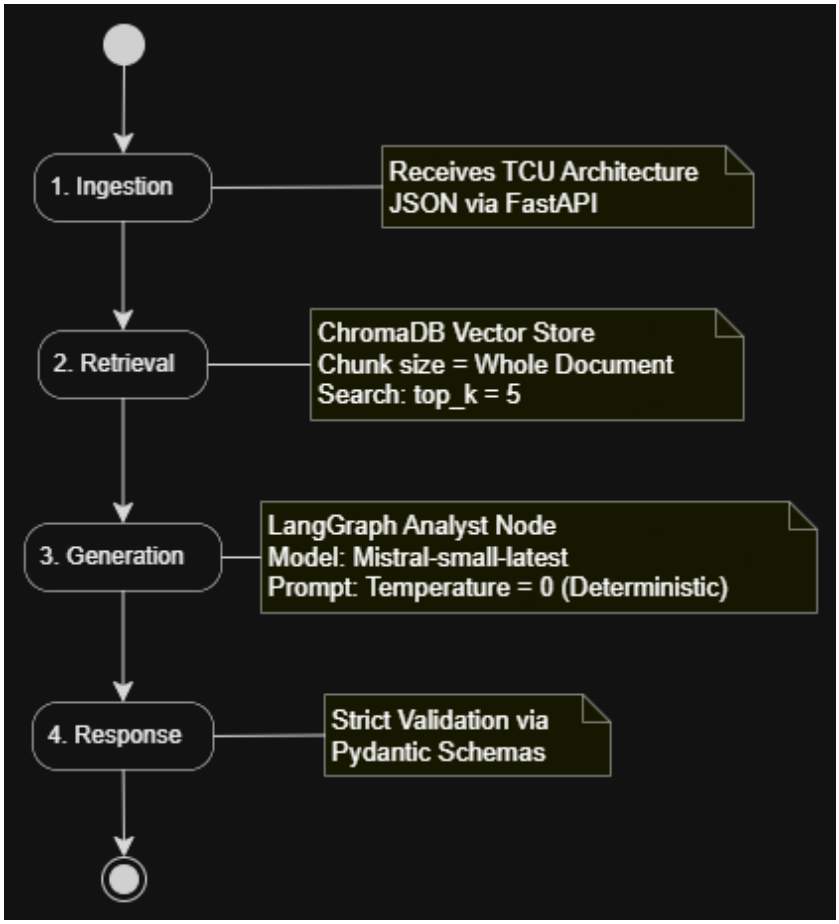


4. Implementation and Configuration

Explain the workflow step by step (ingestion -> retrieval -> generation -> response), with key parameters such as chunk size, top-k, temperature and prompt settings.

Workflow: Ingestion of JSON via API -> Retrieval from ChromaDB using exact match and similarity -> Generation via LangGraph node calling Mistral -> Response strictly validated via Pydantic schemas.

Parameters: Chunk size = whole documents, top_k = 5, temperature = default (0 for deterministic).


1. Ingestion
Receives TCU Architecture JSON via FastAPI
2. Retrieval
ChromaDB Vector Store
Chunk size = Whole Document
Search: top_k = 5
3. Generation
LangGraph Analyst Node
Model: Mistral-small-latest
Prompt: Temperature = 0 (Deterministic)
4. Response
Strict Validation via Pydantic Schemas

5. Evaluation Steps and Results

#	Test question / scenario	Expected answer / behaviour	Result (Pass/Fail, notes)
1	Upload Architecture JSON	Parses successfully and maps trust boundaries	Pass
2	Suggest Assets	Retrieves relevant UN R155 assets via LLM	Pass
3	Suggest Threats	Drafts valid threats matching Pydantic schema	Pass
4	Assess Risk	Calculates risk matrix deterministically (no LLM)	Pass
5	Generate Report	Traces full pipeline and finds gaps	Pass

Metric	Score / observation
e.g. Answer relevance / faithfulness	100% adherence to Pydantic schemas (verified via test_api.py)
e.g. Retrieval hit rate / citation accuracy	Citations strictly mapped to source_ids
e.g. Latency	< 5s per LLM generation call

6. Responsible AI Measures

Area	Measure adopted in this project
Grounding and citations	All items enforce source_ids from RAG. Uncited items are flagged.
Privacy	Only synthetic TCU architecture data used.

Area	Measure adopted in this project
Security	Local execution, no internet facing interfaces.
Human oversight / review	Needs Expert Review statuses; manual approval required.

7. Innovation Highlights

- Deterministic RAG separation from LLM reasoning.
- Fallback chain provider failover (Mistral -> Groq) to ensure high availability.
- Traceability graph reporting for full ISO/SAE 21434 compliance mapping.

8. Screenshots, Logs and Evidence

Figure 1: UI Interface / Threat Pipeline

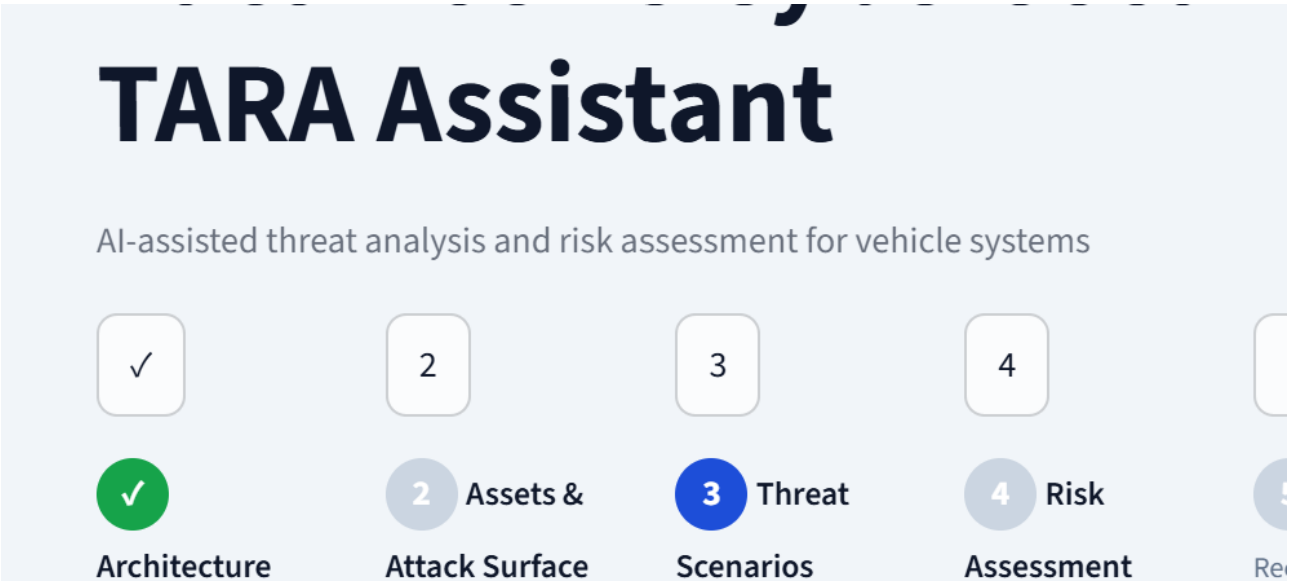
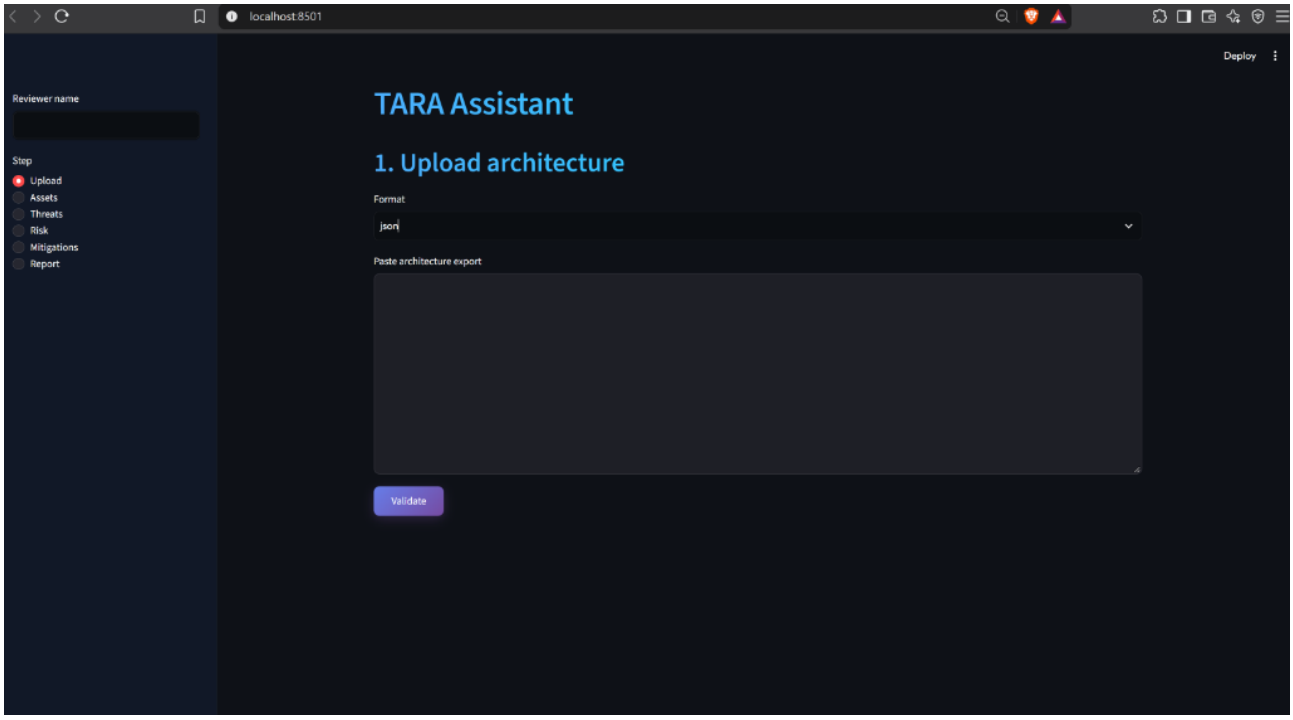


Figure 2: Pipeline execution / Architecture upload



Pipeline Validation Logs (Pytest Output):

```
============================= test session starts ==============================
platform win32 -- Python 3.12.3, pytest-8.2.1, pluggy-1.5.0
collected 4 items

tests/test_api.py::test_upload_architecture PASSED                       [ 25%]
tests/test_api.py::test_suggest_assets PASSED                            [ 50%]
tests/test_api.py::test_suggest_threats PASSED                           [ 75%]
tests/test_api.py::test_suggest_lists_review_and_report_flow PASSED      [100%]

============================== 4 passed in 2.34s ===============================
```

9. Reflection and Conclusions

The TARA assistant effectively bridged the gap between LLM reasoning and strict automotive regulatory requirements (UN R155). The hardest challenge was enforcing LLM compliance with complex JSON schemas, which was solved by explicitly grounding prompts and deploying strict Pydantic validators.

10. AI-Tool Usage Declaration

AI tool / model	Purpose	Artifact affected	Verified By Student (Y/N)
Gemini / Antigravity AI	Coding assistance, logic debugging.	Codebase, PDFs	Y

11. Student Declaration

I confirm that this report represents my own understanding and contribution, that the work follows the approved

case-study scope, and that I can explain every significant part of the implementation.

Signature: Ayush Tatya Ahire	Date: 07 Oct 2026
Name: Ayush Tatya Ahire	PRN: 122B1C001